JOINs

Combine Data Across Multiple Tables

LEARNING OUTCOME

By the end of this module, you will combine data from multiple related tables using all JOIN types, understand the mathematics behind joins, choose the correct join for any business question, and avoid common join mistakes.

Skills You Will Gain
• Use INNER JOIN to match rows in both tables
• Use LEFT JOIN to keep all rows from the left table
• Use RIGHT JOIN and FULL OUTER JOIN
• Use CROSS JOIN for cartesian products
• Write self-joins to compare rows in the same table
• Join more than two tables in one query

■ **JOINs are what make relational databases powerful. 99% of real SQL queries join at least two tables. Customer data + order data + product data -- JOINs bring it all together for analysis.**

SECTION 1

JOIN Types Reference

JOIN Type	Returns	Use When
INNER JOIN	Only rows where condition matches in BOTH tables	You need records that exist in both tables
LEFT JOIN	All rows from LEFT table + matching rows from right (NULLs if no match)	Keep all left rows, optionally include right data
RIGHT JOIN	All rows from RIGHT table + matching left rows (NULLs if no match)	Keep all right rows (rare -- use LEFT JOIN instead)
FULL OUTER JOIN	All rows from BOTH tables with NULLs where no match	Find rows that exist in either but not both
CROSS JOIN	Every combination of rows (m x n rows)	Generate all combinations (size grids, test data)
SELF JOIN	Join table to itself using aliases	Compare rows within same table (hierarchy, pairs)

SECTION 2

INNER JOIN -- The Most Common

```
-- INNER JOIN: only customers who have orders SELECT c.customer_id, c.first_name, c.city,
o.order_id, o.order_date, o.total_amount FROM customers c INNER JOIN orders o ON c.customer_id =
o.customer_id; -- JOIN (without INNER keyword) is INNER JOIN by default SELECT c.first_name,
o.total_amount FROM customers c JOIN orders o ON c.customer_id = o.customer_id; -- Three-table JOIN
SELECT c.first_name, c.city, o.order_date, p.product_name, oi.quantity, oi.unit_price, oi.quantity
* oi.unit_price AS line_total FROM customers c JOIN orders o ON c.customer_id = o.customer_id JOIN
order_items oi ON o.order_id = oi.order_id JOIN products p ON oi.product_id = p.product_id WHERE
o.status = 'delivered' ORDER BY line_total DESC;
```

SECTION 3

LEFT JOIN and FULL OUTER JOIN

```
-- LEFT JOIN: ALL customers, even those without orders SELECT c.customer_id, c.first_name,
COUNT(o.order_id) AS order_count, -- 0 for customers with no orders SUM(o.total_amount) AS
total_spent -- NULL for no orders FROM customers c LEFT JOIN orders o ON c.customer_id =
o.customer_id GROUP BY c.customer_id, c.first_name; -- Find customers who have NEVER ordered SELECT
c.customer_id, c.first_name, c.email FROM customers c LEFT JOIN orders o ON c.customer_id =
o.customer_id WHERE o.order_id IS NULL; -- NULL means no matching order -- Find products never
ordered SELECT p.product_id, p.product_name FROM products p LEFT JOIN order_items oi ON
p.product_id = oi.product_id WHERE oi.order_id IS NULL; -- FULL OUTER JOIN: customers without
orders AND orders without customers SELECT COALESCE(c.customer_id, o.customer_id) AS id,
c.first_name, o.order_id, o.total_amount FROM customers c FULL OUTER JOIN orders o ON c.customer_id
= o.customer_id; -- Emulate FULL OUTER JOIN in MySQL (no FULL OUTER JOIN support) SELECT
c.first_name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id UNION
SELECT c.first_name, o.order_id FROM customers c RIGHT JOIN orders o ON
c.customer_id=o.customer_id;
```

SECTION 4

Self JOIN and CROSS JOIN

```
-- SELF JOIN: compare employees to their managers (same table) SELECT e.employee_id, e.name AS employee_name, e.salary AS employee_salary, m.name AS manager_name, m.salary AS manager_salary FROM employees e JOIN employees m ON e.manager_id = m.employee_id; -- Self join: find pairs of customers in same city SELECT a.customer_id AS customer1, b.customer_id AS customer2, a.city FROM customers a JOIN customers b ON a.city = b.city AND a.customer_id < b.customer_id; -- avoid duplicates -- CROSS JOIN: all size/colour combinations for product variants SELECT s.size_name, c.colour_name, CONCAT(s.size_name, ' / ', c.colour_name) AS variant FROM sizes s CROSS JOIN colours c; -- Non-equi JOIN: join on a range condition SELECT c.first_name, c.annual_spend, t.tier_name FROM customers c JOIN customer_tiers t ON c.annual_spend BETWEEN t.min_spend AND t.max_spend;
```

■ PRACTICE Q & A

Q1. What is the difference between INNER JOIN and LEFT JOIN?

A: *INNER JOIN returns only rows where condition matches in BOTH tables. LEFT JOIN returns ALL rows from the left table plus matching rows from right -- NULL where no match. Use LEFT JOIN to keep all records from the primary table.*

Q2. How do you find rows that exist in the left table but NOT in the right table?

A: *Use LEFT JOIN and filter WHERE right_table.primary_key IS NULL. This is the anti-join pattern: find customers with no orders, products never ordered.*

Q3. What happens when you JOIN without an ON condition?

A: *You get a CROSS JOIN -- every combination of rows from both tables. m rows x n rows = m*n rows. Usually a mistake. Always specify an ON condition for intended joins.*

Q4. What is a self join?

A: *Joining a table to itself using two different aliases. Used for hierarchical data (employees and managers), finding pairs that satisfy conditions within the same table, or comparing rows to other rows in the same table.*

Q5. What is a non-equi join?

A: *A JOIN where the condition uses something other than = (e.g., BETWEEN, >, <). Example: joining customers to tier table based on whether spend falls within min/max range.*

■ JOINs Cheat Sheet	
`INNER JOIN t ON a.id=b.id`	Only matching rows
`LEFT JOIN t ON a.id=b.id`	All left + matched right
`RIGHT JOIN t ON a.id=b.id`	All right + matched left
`FULL OUTER JOIN`	All rows from both tables
`CROSS JOIN`	Every combination (m x n)
`WHERE right.id IS NULL`	Anti-join: left only
`WHERE left.id IS NULL`	Anti-join: right only
`t1 a JOIN t1 b ON ...`	Self join (same table)
`ON a.val BETWEEN b.lo AND b.hi`	Non-equi join
`JOIN on multiple cols`	ON a.x=b.x AND a.y=b.y
`COUNT(o.id) after LEFT JOIN`	0 for unmatched rows
`COALESCE(a.col, b.col)`	First non-NULL from either